Porter Delivery Time Estimation

Context:

Porter is India's Largest Marketplace for Intra-City Logistics. Leader in the country's $40 billion intra-city logistics market, Porter strives to improve the lives of 1,50,000+ driver-partners by providing them with consistent earning & independence. Currently, the company has serviced 5+ million customers.

Porter works with a wide range of restaurants for delivering their items directly to the people.

Problem Statement

Porter has a number of delivery partners available for delivering the food, from various restaurants and wants to get an estimated delivery time that it can provide the customers on the basis of what they are ordering, from where and also the delivery partners.

This dataset has the required data to train a regression model that will do the delivery time estimation, based on all those features

What is expected?

As a data scientist at Porter, your task is to analyze the dataset to accurately predict delivery times for different orders. Your primary goal is to build a regression model using neural networks, evaluate its performance, and provide insights for optimizing delivery operations.

1. Importing Libraries

In [2]:
```
# for reading and handling the data
import numpy as np
import pandas as pd

# for visualizinng and analyzing it
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# data preprocessing
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# random forest model training
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error
from sklearn.ensemble import RandomForestRegressor

# NN training
from tensorflow.keras import Model
from tensorflow.keras import Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Dense,Dropout,BatchNormalization,LeakyReLU
from sklearn.model_selection import train_test_split
from tensorflow.keras.losses import MeanSquaredLogarithmicError
from tensorflow.keras.losses import MeanSquaredError
from tensorflow.keras.losses import MeanAbsolutePercentageError

from tensorflow.keras.metrics import MeanAbsolutePercentageError
from tensorflow.keras.metrics import RootMeanSquaredError
from tensorflow.keras.metrics import MeanAbsoluteError
from tensorflow.keras.optimizers import SGD,Adam
```

2. Loading Dataset

In [3]:
```
df = pd.read_csv('porter_data.csv')
df.head()
```

Out[3]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	1.0	4	3441	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	2.0	1	1900	1	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	3.0	4	4771	3	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	1.0	1	1525	1	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	1.0	2	3620	2	

In [4]:
```
df.shape
```

Out[4]:
```
(175777, 14)
```

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   created_at                                    175777 non-null  object
 2   actual_delivery_time                          175777 non-null  object
 3   store_primary_category                        175777 non-null  int64
 4   order_protocol                                175777 non-null  float64
 5   total_items                                   175777 non-null  int64
 6   subtotal                                      175777 non-null  int64
 7   num_distinct_items                            175777 non-null  int64
 8   min_item_price                                175777 non-null  int64
 9   max_item_price                                175777 non-null  int64
 10  total_onshift_dashers                         175777 non-null  float64
 11  total_busy_dashers                            175777 non-null  float64
 12  total_outstanding_orders                      175777 non-null  float64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  float64
dtypes: float64(6), int64(6), object(2)
memory usage: 18.8+ MB
```

Data preprocessing

Feature engineering

We have the time at which the order was placed and time at which it was delivered, so we will create a new column for time taken in delivery and that will be our target column

Calculating time taken in delivery by subtracting the order timestamp from delivery timestamp

The time stamps that we have now are in object format and need to be convertd to datetime format for easily working with them as intended. The pandas datetime function checks if the data is in correct format for it and also understands the order of the data and converts accordingly

In [6]:
```
df['created_at'] = pd.to_datetime(df['created_at'])
df['actual_delivery_time'] = pd.to_datetime(df['actual_delivery_time'])
```

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   created_at                                    175777 non-null  datetime64[ns]
 2   actual_delivery_time                          175777 non-null  datetime64[ns]
 3   store_primary_category                        175777 non-null  int64
 4   order_protocol                                175777 non-null  float64
 5   total_items                                   175777 non-null  int64
 6   subtotal                                      175777 non-null  int64
 7   num_distinct_items                            175777 non-null  int64
 8   min_item_price                                175777 non-null  int64
 9   max_item_price                                175777 non-null  int64
 10  total_onshift_dashers                         175777 non-null  float64
 11  total_busy_dashers                            175777 non-null  float64
 12  total_outstanding_orders                      175777 non-null  float64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  float64
dtypes: datetime64[ns](2), float64(6), int64(6)
memory usage: 18.8 MB
```

In [8]:
```
df['time_taken'] = df['actual_delivery_time'] - df['created_at']
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 15 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   created_at                                    175777 non-null  datetime64[ns]
 2   actual_delivery_time                          175777 non-null  datetime64[ns]
 3   store_primary_category                        175777 non-null  int64
 4   order_protocol                                175777 non-null  float64
 5   total_items                                   175777 non-null  int64
 6   subtotal                                      175777 non-null  int64
 7   num_distinct_items                            175777 non-null  int64
 8   min_item_price                                175777 non-null  int64
 9   max_item_price                                175777 non-null  int64
 10  total_onshift_dashers                         175777 non-null  float64
 11  total_busy_dashers                            175777 non-null  float64
 12  total_outstanding_orders                      175777 non-null  float64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  float64
 14  time_taken                                    175777 non-null  timedelta64[ns]
dtypes: datetime64[ns](2), float64(6), int64(6), timedelta64[ns](1)
memory usage: 20.1 MB
```

In [9]:
```
df.head()
```

Out[9]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	1.0	4	3441	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	2.0	1	1900	1	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	3.0	4	4771	3	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	1.0	1	1525	1	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	1.0	2	3620	2	

Now that we have our time taken for the delivery we can convert it to minutes and that will be our target variable to train the models

The timedelta is a datatype that stores the time difference and it is better we convert it to float and converting to minute does that as well

In [10]:
```
df['time_taken_mins'] = pd.to_timedelta(df['time_taken'])/pd.Timedelta('60s')
df.head()
```

Out[10]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	1.0	4	3441	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	2.0	1	1900	1	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	3.0	4	4771	3	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	1.0	1	1525	1	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	1.0	2	3620	2	

We can also extract the hour at which the order was placed and which day of the week it was

In [11]:

```
df['hour'] = df['created_at'].dt.hour
df['day'] = df['created_at'].dt.dayofweek
df.head()
```

Out[11]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	1.0	4	3441	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	2.0	1	1900	1	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	3.0	4	4771	3	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	1.0	1	1525	1	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	1.0	2	3620	2	

Dropping the columns that are no longer required

In [12]:
```
df.drop(['time_taken','created_at','actual_delivery_time'],axis=1,inplace=True)
```

Checking null values in the data

In [13]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 15 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   store_primary_category                        175777 non-null  int64
 2   order_protocol                                175777 non-null  float64
 3   total_items                                   175777 non-null  int64
 4   subtotal                                      175777 non-null  int64
 5   num_distinct_items                            175777 non-null  int64
 6   min_item_price                                175777 non-null  int64
 7   max_item_price                                175777 non-null  int64
 8   total_onshift_dashers                         175777 non-null  float64
 9   total_busy_dashers                            175777 non-null  float64
 10  total_outstanding_orders                      175777 non-null  float64
 11  estimated_store_to_consumer_driving_duration  175777 non-null  float64
 12  time_taken_mins                               175777 non-null  float64
 13  hour                                          175777 non-null  int32
 14  day                                           175777 non-null  int32
dtypes: float64(7), int32(2), int64(6)
memory usage: 18.8 MB
```

In [14]:
```
df.isna().sum()
```

Out[14]:

	0
market_id	0
store_primary_category	0
order_protocol	0
total_items	0
subtotal	0
num_distinct_items	0
min_item_price	0
max_item_price	0
total_onshift_dashers	0
total_busy_dashers	0
total_outstanding_orders	0
estimated_store_to_consumer_driving_duration	0
time_taken_mins	0
hour	0
day	0

dtype: int64

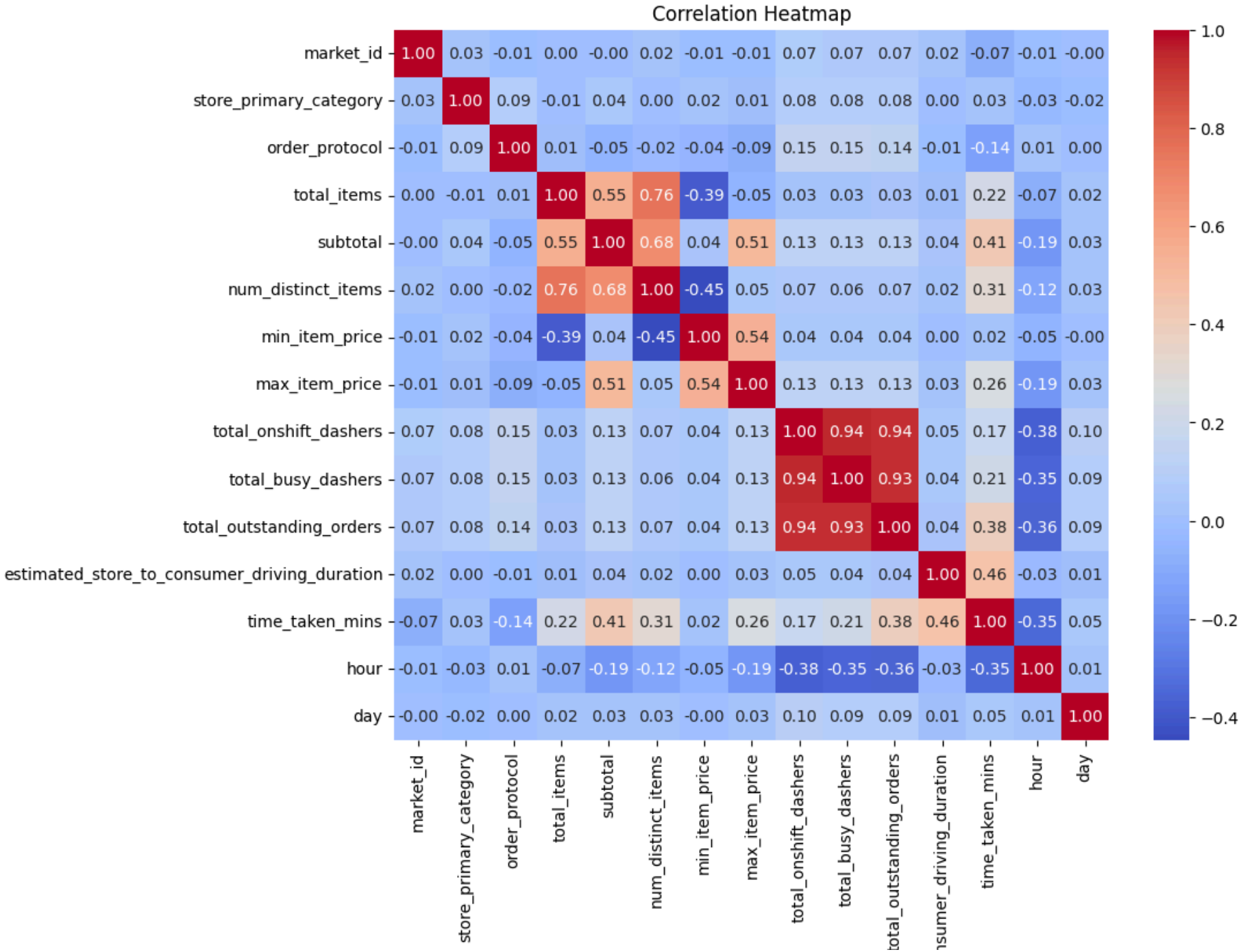
Plotting correlation to get an idea of the data

In [15]:

```
correlation_matrix = df.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
```

```
plt.title('Correlation Heatmap')
plt.show()
```


Correlation Heatmap
market_id 1.00 0.03 -0.01 0.00 -0.00 0.02 -0.01 -0.01 0.07 0.07 0.07 0.02 -0.07 -0.01 -0.00
store_primary_category 0.03 1.00 0.09 -0.01 0.04 0.00 0.02 0.01 0.08 0.08 0.08 0.00 0.03 -0.03 -0.02
order_protocol -0.01 0.09 1.00 0.01 -0.05 -0.02 -0.04 -0.09 0.15 0.15 0.14 -0.01 -0.14 0.01 0.00
total_items 0.00 -0.01 0.01 1.00 0.55 0.76 -0.39 -0.05 0.03 0.03 0.03 0.01 0.22 -0.07 0.02
subtotal -0.00 0.04 -0.05 0.55 1.00 0.68 0.04 0.51 0.13 0.13 0.13 0.04 0.41 -0.19 0.03
num_distinct_items 0.02 0.00 -0.02 0.76 0.68 1.00 -0.45 0.05 0.07 0.06 0.07 0.02 0.31 -0.12 0.03
min_item_price -0.01 0.02 -0.04 -0.39 0.04 -0.45 1.00 0.54 0.04 0.04 0.04 0.00 0.02 -0.05 -0.00
max_item_price -0.01 0.01 -0.09 -0.05 0.51 0.05 0.54 1.00 0.13 0.13 0.13 0.03 0.26 -0.19 0.03
total_onshift_dashers 0.07 0.08 0.15 0.03 0.13 0.07 0.04 0.13 1.00 0.94 0.94 0.05 0.17 -0.38 0.10
total_busy_dashers 0.07 0.08 0.15 0.03 0.13 0.06 0.04 0.13 0.94 1.00 0.93 0.04 0.21 -0.35 0.09
total_outstanding_orders 0.07 0.08 0.14 0.03 0.13 0.07 0.04 0.13 0.94 0.93 1.00 0.04 0.38 -0.36 0.09
estimated_store_to_consumer_driving_duration 0.02 0.00 -0.01 0.01 0.04 0.02 0.00 0.03 0.05 0.04 0.04 1.00 0.46 -0.03 0.01
time_taken_mins -0.07 0.03 -0.14 0.22 0.41 0.31 0.02 0.26 0.17 0.21 0.38 0.46 1.00 -0.35 0.05
hour -0.01 -0.03 0.01 -0.07 -0.19 -0.12 -0.05 -0.19 -0.38 -0.35 -0.36 -0.03 -0.35 1.00 0.01
day -0.00 -0.02 0.00 0.02 0.03 0.03 -0.00 0.03 0.10 0.09 0.09 0.01 0.05 0.01 1.00
market_id
store_primary_category
order_protocol
total_items
subtotal
num_distinct_items
min_item_price
max_item_price
total_onshift_dashers
total_busy_dashers
total_outstanding_orders
nsumer_driving_duration
time_taken_mins
hour
day
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

estimated_store_to_cor

we have one categorical column which we will change to integer for model

In [16]:
```
df['store_primary_category']=df['store_primary_category'].astype('category').cat.codes
```

In [17]:
```
df.info()
```

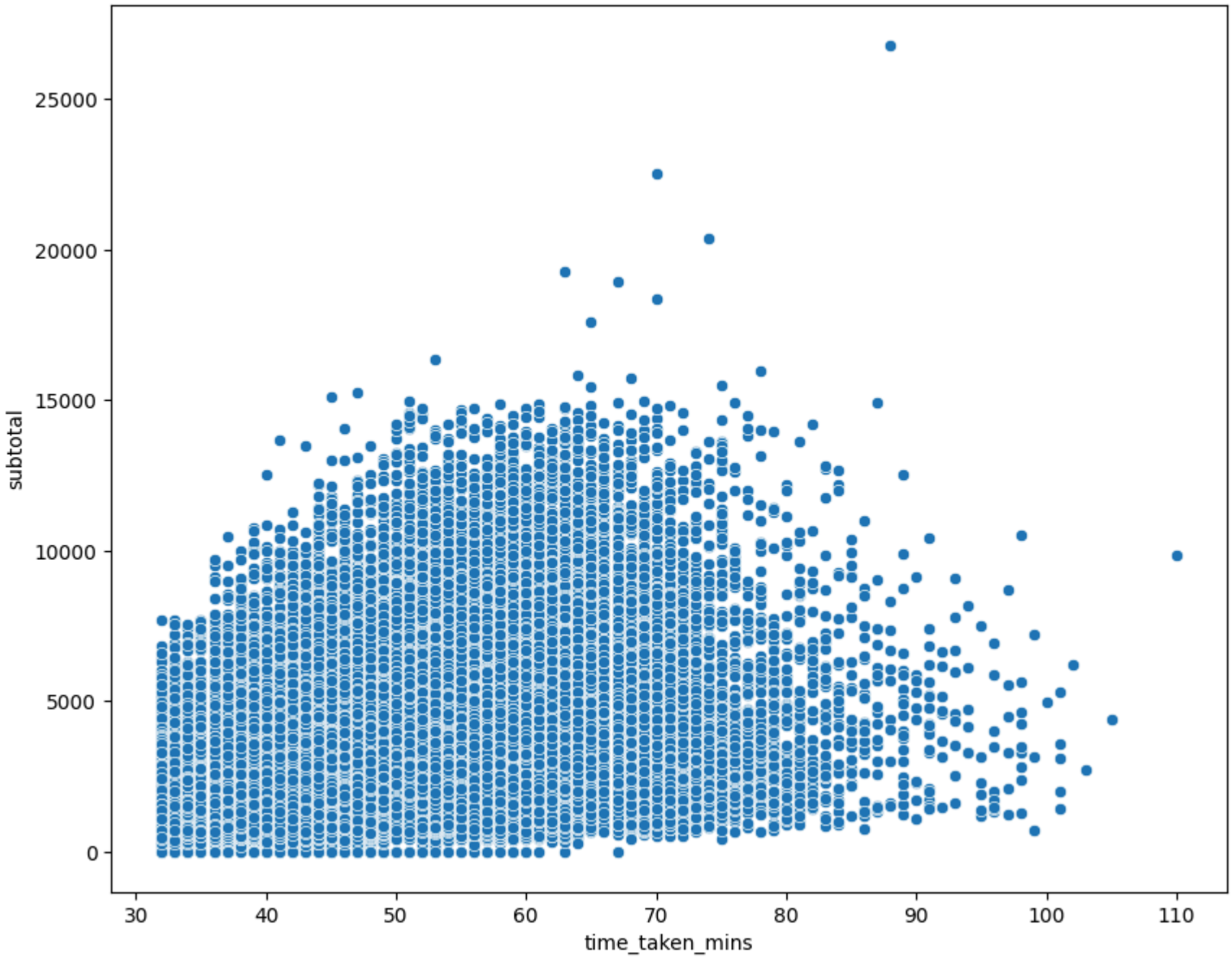
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 15 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   store_primary_category                        175777 non-null  int8
 2   order_protocol                                175777 non-null  float64
 3   total_items                                   175777 non-null  int64
 4   subtotal                                      175777 non-null  int64
 5   num_distinct_items                            175777 non-null  int64
 6   min_item_price                                175777 non-null  int64
 7   max_item_price                                175777 non-null  int64
 8   total_onshift_dashers                         175777 non-null  float64
 9   total_busy_dashers                            175777 non-null  float64
 10  total_outstanding_orders                      175777 non-null  float64
 11  estimated_store_to_consumer_driving_duration  175777 non-null  float64
 12  time_taken_mins                               175777 non-null  float64
 13  hour                                          175777 non-null  int32
 14  day                                           175777 non-null  int32
dtypes: float64(7), int32(2), int64(5), int8(1)
memory usage: 17.6 MB
```

In [17]:

Data Visualization and Cleaning

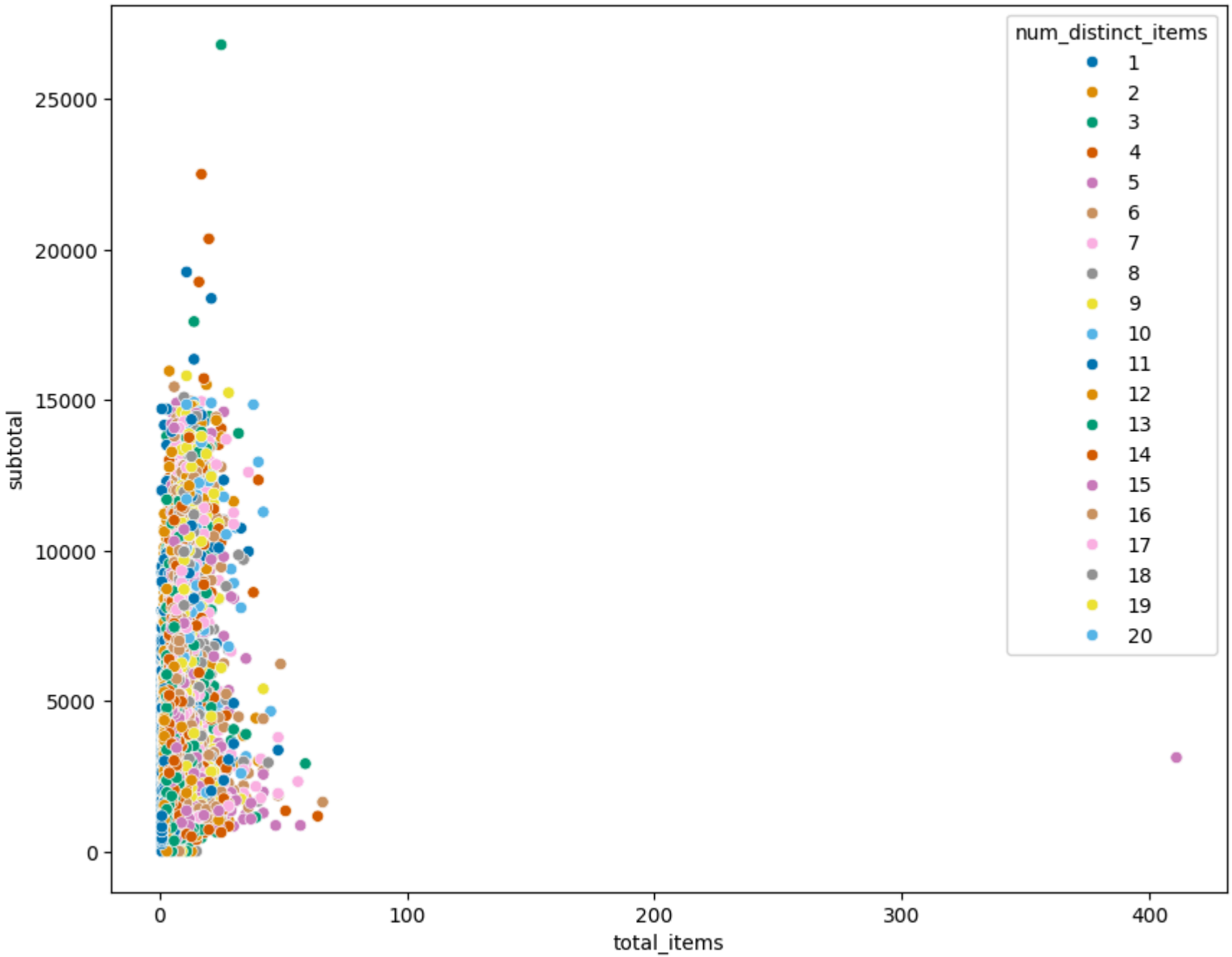
In [18]:
```
plt.figure(figsize=(10, 8))
sns.scatterplot(x='time_taken_mins', y='subtotal', data=df)
```

Out[18]:
```
<Axes: xlabel='time_taken_mins', ylabel='subtotal'>
```


25000
20000
15000
10000
5000
0
subtotal
30
40
50
60
70
80
90
100
110
time_taken_mins

In [19]:
```
plt.figure(figsize=(10, 8))
sns.scatterplot(x='total_items',y='subtotal',hue='num_distinct_items',palette='colorblind',data=df)
```

Out[19]:
```
<Axes: xlabel='total_items', ylabel='subtotal'>
```


num_distinct_items
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
subtotal
25000
20000
15000
10000
5000
0
total_items
0
100
200
300
400

In [20]:
```
from sklearn.neighbors import LocalOutlierFactor
model1=LocalOutlierFactor()
df['lof_anomaly_score'] = model1.fit_predict(df)
```

In [21]:
```
print("number of outliers : ",(len(df.loc[(df['lof_anomaly_score'] == -1)])))
df=df.loc[(df['lof_anomaly_score'] == 1)]
```

```
number of outliers :  831
```

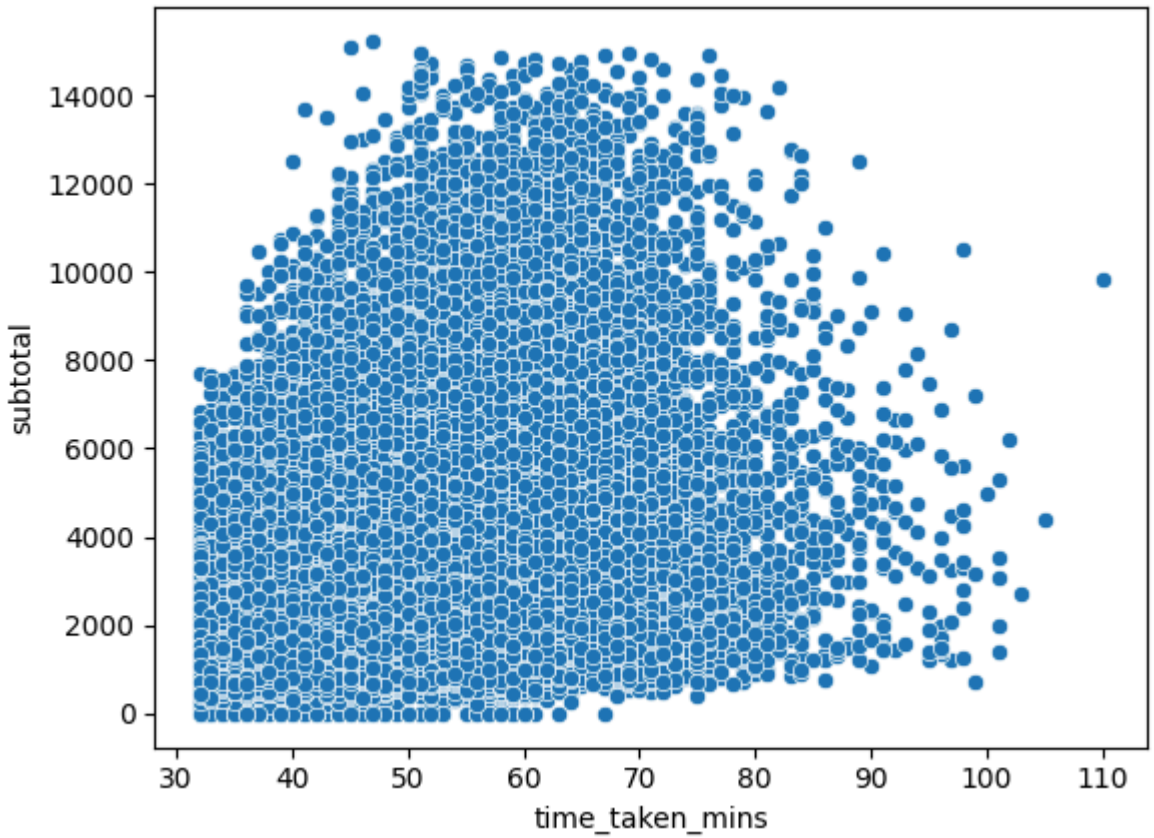
In [22]:
```
df.drop(['lof_anomaly_score'],axis=1,inplace=True)
```

In [23]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 174946 entries, 0 to 175776
Data columns (total 15 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     174946 non-null  float64
 1   store_primary_category                        174946 non-null  int8
 2   order_protocol                                174946 non-null  float64
 3   total_items                                   174946 non-null  int64
 4   subtotal                                      174946 non-null  int64
 5   num_distinct_items                            174946 non-null  int64
 6   min_item_price                                174946 non-null  int64
 7   max_item_price                                174946 non-null  int64
 8   total_onshift_dashers                         174946 non-null  float64
 9   total_busy_dashers                            174946 non-null  float64
 10  total_outstanding_orders                      174946 non-null  float64
 11  estimated_store_to_consumer_driving_duration  174946 non-null  float64
 12  time_taken_mins                               174946 non-null  float64
 13  hour                                          174946 non-null  int32
 14  day                                           174946 non-null  int32
dtypes: float64(7), int32(2), int64(5), int8(1)
memory usage: 18.9 MB
```

In [24]:
```
sns.scatterplot(x='time_taken_mins',y='subtotal',data=df)
```

Out[24]:
```
<Axes: xlabel='time_taken_mins', ylabel='subtotal'>
```



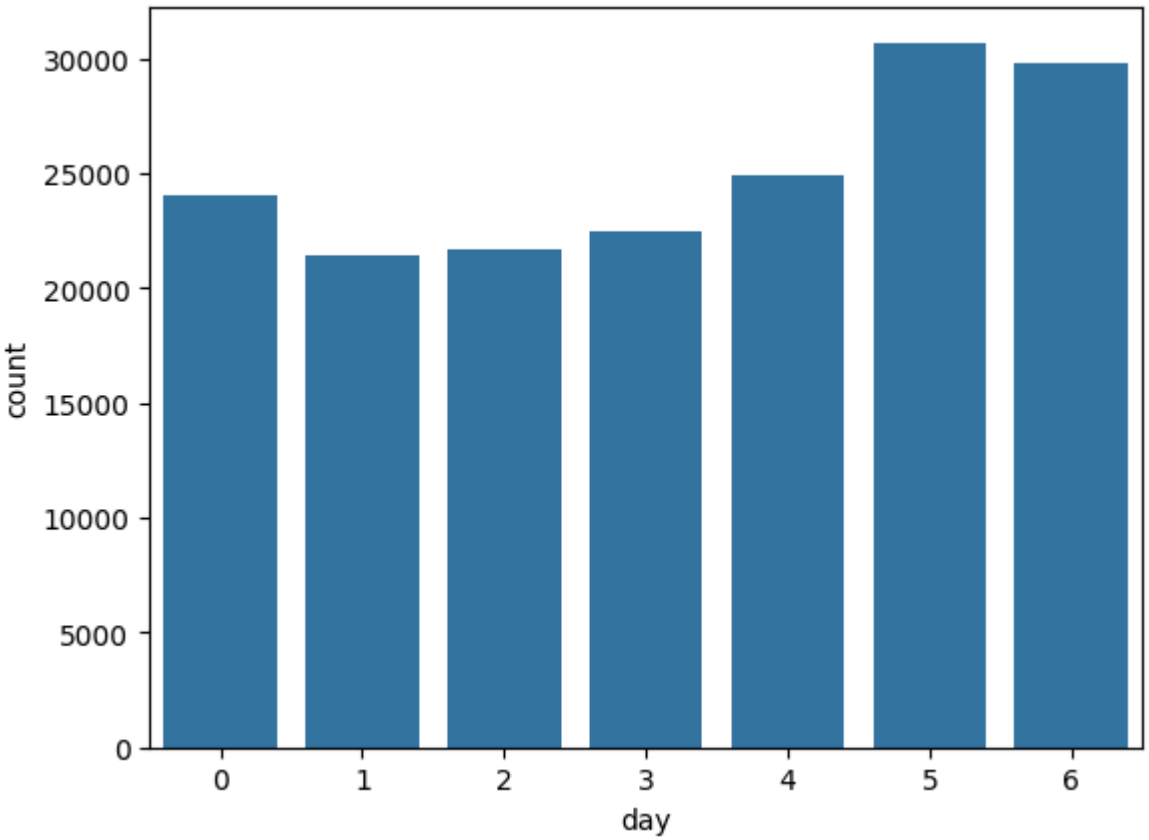
We can see that after removing outliers our data is looking better

In [25]:
```
df.columns
```

Out[25]:
```
Index(['market_id', 'store_primary_category', 'order_protocol', 'total_items',
       'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
       'total_onshift_dashers', 'total_busy_dashers',
       'total_outstanding_orders',
       'estimated_store_to_consumer_driving_duration', 'time_taken_mins',
       'hour', 'day'],
      dtype='object')
```

In [26]:
```
sns.countplot(x=df.day)
```

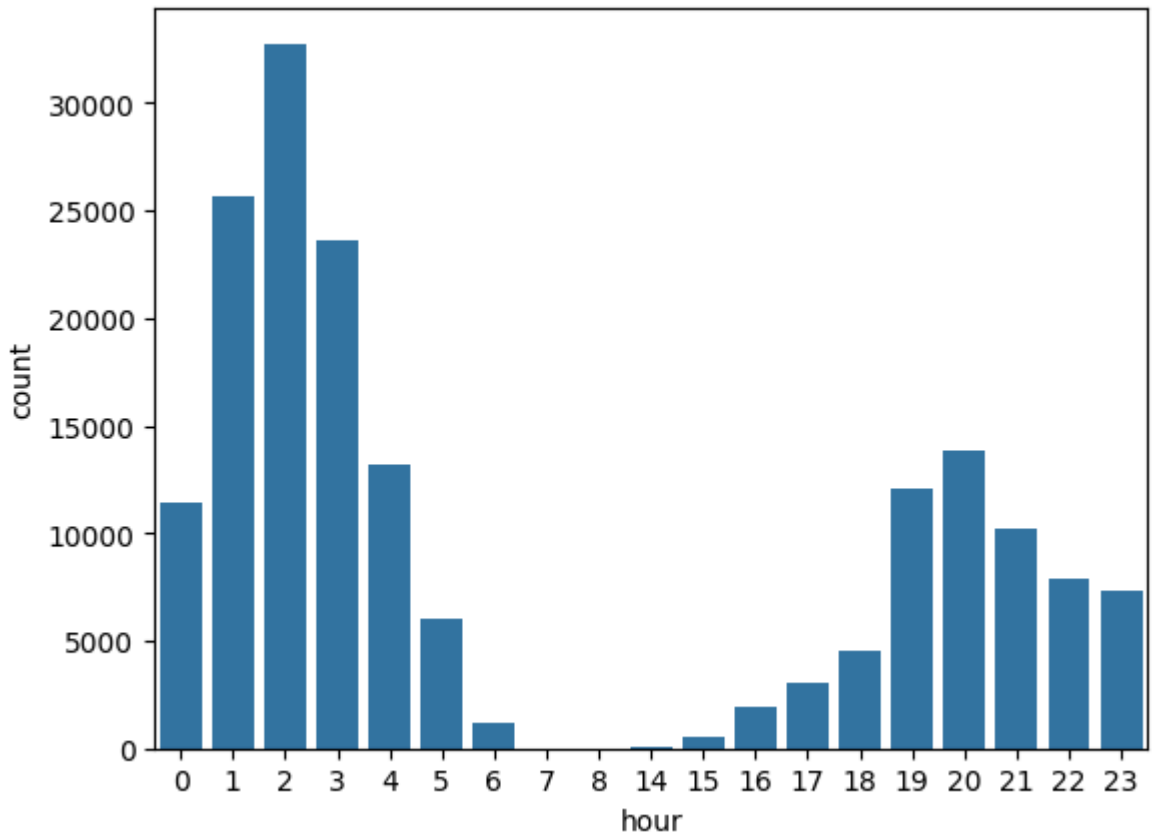
Out[26]: <Axes: xlabel='day', ylabel='count'>



a little more orders on the weekends

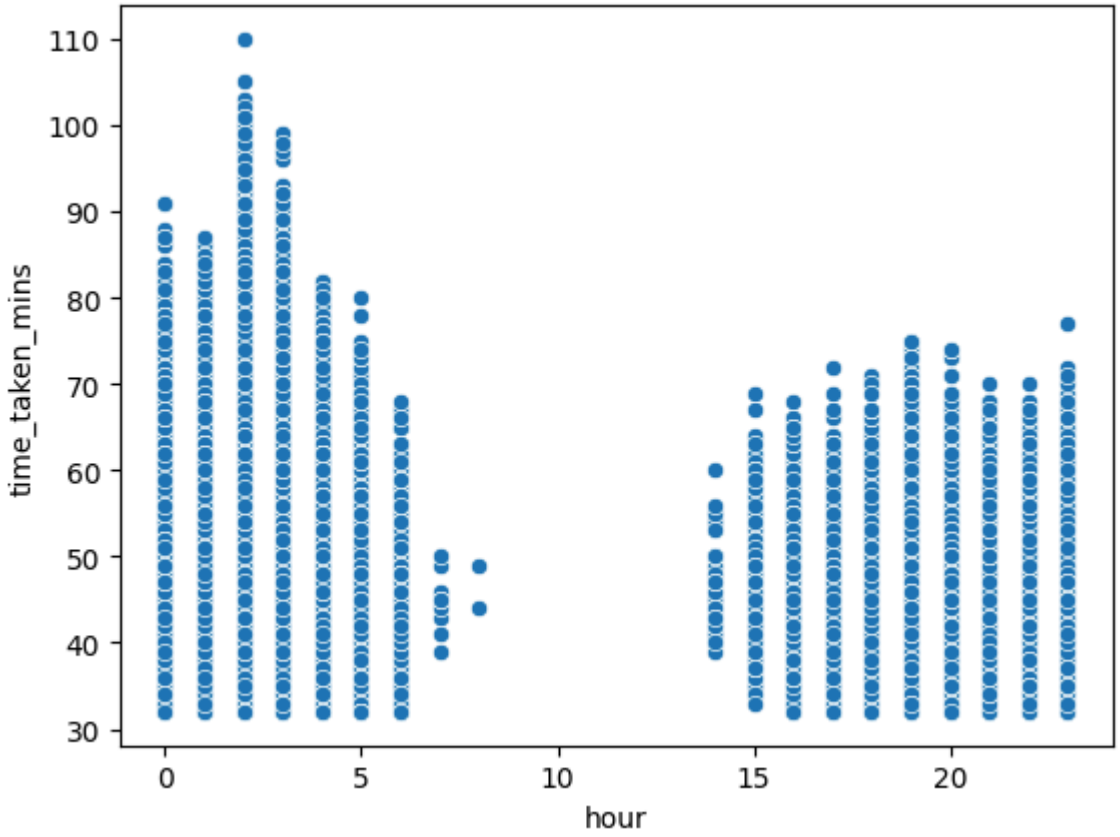
In [27]:
```
sns.countplot(x=df.hour)
```

Out[27]: <Axes: xlabel='hour', ylabel='count'>



In [28]:
```
sns.scatterplot(x='hour',y='time_taken_mins',data=df)
```

Out[28]:
```
<Axes: xlabel='hour', ylabel='time_taken_mins'>
```



Data Splitting and Modelling

In [29]:
```
y = df['time_taken_mins']
x=df.drop(['time_taken_mins'],axis=1)
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

In [30]:
```
x.head()
```

Out[30]:

	market_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onsl
0	1.0	4	1.0	4	3441	4	557	1239	
1	2.0	46	2.0	1	1900	1	1400	1400	
2	2.0	36	3.0	4	4771	3	820	1604	
3	1.0	38	1.0	1	1525	1	1525	1525	
4	1.0	38	1.0	2	3620	2	1425	2195	

In [31]:

```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(139956, 14)
(34990, 14)
(139956,)
(34990,)
```

Random Forest

In [32]:

```
regressor = RandomForestRegressor()
regressor.fit(X_train, y_train)
```

Out[32]:

```
▾ RandomForestRegressor
RandomForestRegressor()
```

In [33]:

```
prediction=regressor.predict(X_test)
mse=mean_squared_error(y_test,prediction)
rmse=mse**.5
print("mse : ",mse)
print("rmse : ",rmse)
```

```
mae=mean_absolute_error(y_test,prediction)
print("mae : ",mae)
```

```
mse :  3.235961940554444
rmse :  1.7988779671101773
mae :  1.287867676478994
```

In [34]:
```
r2_score(y_test,prediction)
```

Out[34]:
```
0.9623902018162022
```

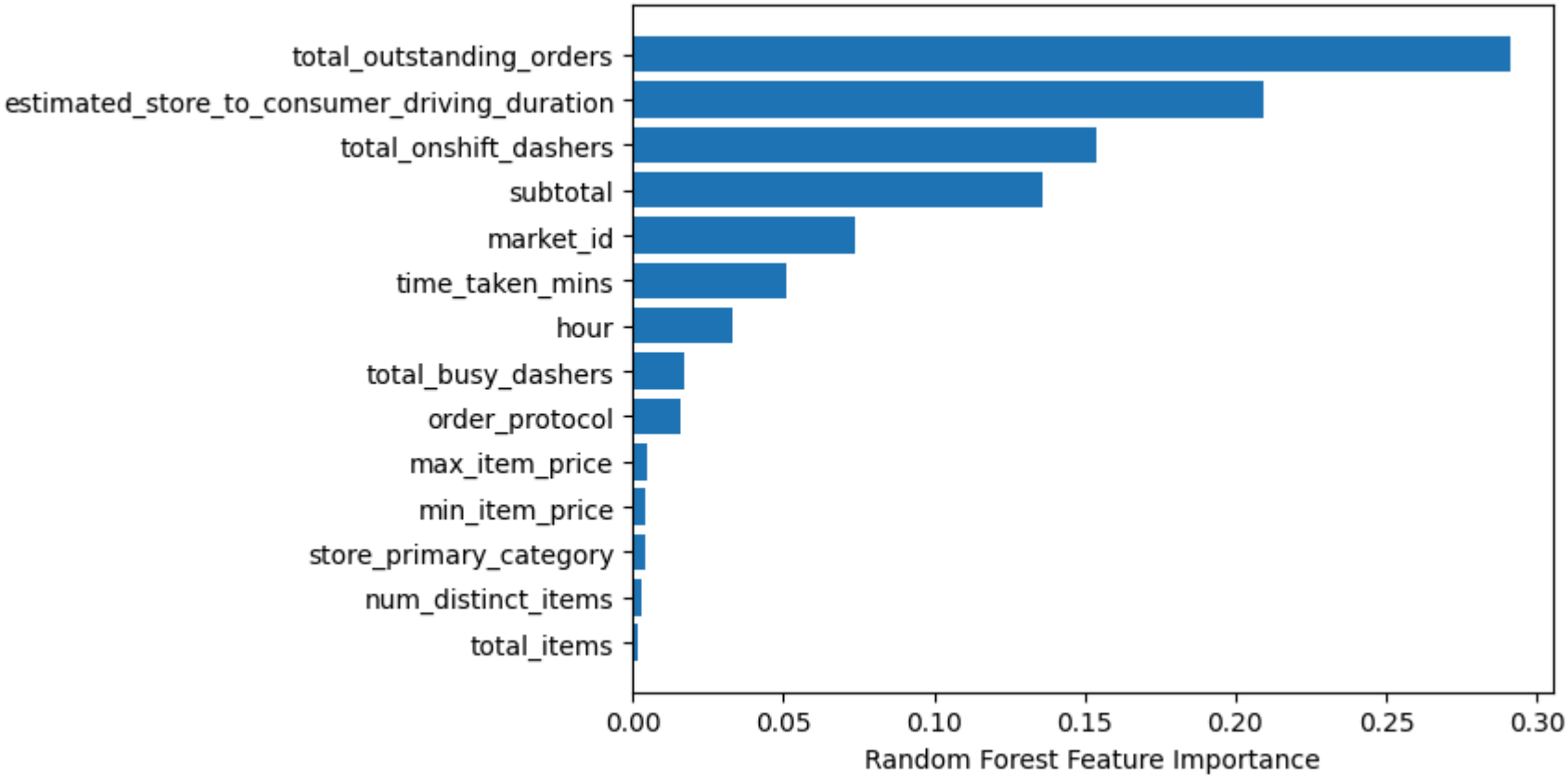
In [35]:
```
def MAPE(Y_actual,Y_Predicted):
    mape=np.mean(np.abs((Y_actual - Y_Predicted)/Y_actual))*100
    return mape

print("mape : ",MAPE(y_test,prediction))
```

```
mape :  2.7746982515888083
```

In [36]:
```
sorted_idx = regressor.feature_importances_.argsort()
plt.barh(df.columns[sorted_idx], regressor.feature_importances_[sorted_idx])
plt.xlabel("Random Forest Feature Importance")
```

Out[36]:
```
Text(0.5, 0, 'Random Forest Feature Importance')
```



Neural Network

Scalling the data to feed before neural network

In [37]:
```
from sklearn import preprocessing
scaler = preprocessing.MinMaxScaler()
x_scaled=scaler.fit_transform(x)
X_train,X_test,y_train,y_test=train_test_split(x_scaled,y,test_size=0.2,random_state=42)
```

We will build a simple neural network to train our regression model it is a sequential model with three layers,

we have kept the number of nodes in the first layers equal to the number of input columns, and for the subsequent layers 512,1024,256, which can we changed or experimented with

the activation for the layers is kept as relu because it is a great non linear activation function that works for most cases, we could have used leaky relu if we see gradient vanishing.

the last layer has one node because it will give the single result that is our delivery time and the activation function for that should be linear

In [38]:
```
model=Sequential()
model.add(Dense(14,kernel_initializer='normal',activation='relu'))
model.add(Dense(512,activation='relu'))
model.add(Dense(1024,activation='relu'))
model.add(Dense(256,activation='relu'))
model.add(Dense(1,activation='linear'))
```

we use adam optimizer which is extention to classic schostic gradient descent(SGD) algorithm, but handles much of its drawbacks

Stochastic gradient descent maintains a single learning rate (termed alpha) for all weight updates and the learning rate does not change during training.

In [39]:
```
adam = Adam(learning_rate=0.01)
model.compile(loss='mse',optimizer=adam,metrics=['mse','mae'])
history = model.fit(X_train,y_train,epochs=30,batch_size=512,validation_split=0.2)
```

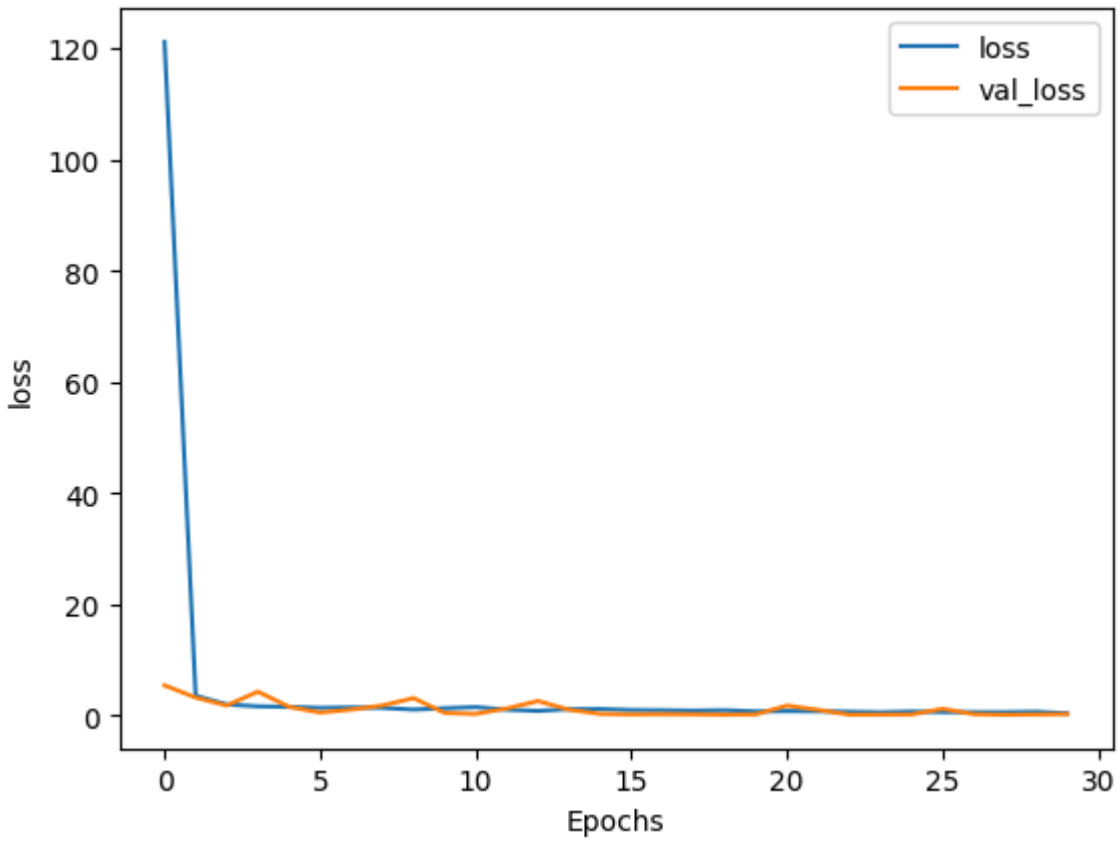
```
Epoch 1/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 18s 70ms/step - loss: 357.8820 - mae: 12.9547 - mse: 357.8820 - val_loss: 5.4229 - val_mae: 1.7102
- val_mse: 5.4229
Epoch 2/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 21s 72ms/step - loss: 4.6373 - mae: 1.5984 - mse: 4.6373 - val_loss: 3.2281 - val_mae: 1.5339 - va
l_mse: 3.2281
Epoch 3/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 22s 79ms/step - loss: 2.3249 - mae: 1.2079 - mse: 2.3249 - val_loss: 1.8018 - val_mae: 1.1016 - va
l_mse: 1.8018
Epoch 4/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 16s 71ms/step - loss: 1.6225 - mae: 0.9898 - mse: 1.6225 - val_loss: 4.2904 - val_mae: 1.9064 - va
l_mse: 4.2904
Epoch 5/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 20s 70ms/step - loss: 1.6274 - mae: 0.9989 - mse: 1.6274 - val_loss: 1.5679 - val_mae: 1.0646 - va
l_mse: 1.5679
Epoch 6/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 21s 74ms/step - loss: 1.3229 - mae: 0.8826 - mse: 1.3229 - val_loss: 0.5345 - val_mae: 0.5485 - va
l_mse: 0.5345
Epoch 7/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 21s 74ms/step - loss: 1.4632 - mae: 0.9445 - mse: 1.4632 - val_loss: 1.0702 - val_mae: 0.8671 - va
l_mse: 1.0702
Epoch 8/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 15s 69ms/step - loss: 1.2574 - mae: 0.9013 - mse: 1.2574 - val_loss: 1.7940 - val_mae: 1.1874 - va
l_mse: 1.7940
Epoch 9/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 16s 74ms/step - loss: 1.1350 - mae: 0.8417 - mse: 1.1350 - val_loss: 3.1288 - val_mae: 1.6558 - va
l_mse: 3.1288
Epoch 10/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 22s 80ms/step - loss: 1.6977 - mae: 1.0655 - mse: 1.6977 - val_loss: 0.4645 - val_mae: 0.5531 - va
l_mse: 0.4645
Epoch 11/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 19s 74ms/step - loss: 1.3938 - mae: 0.9251 - mse: 1.3938 - val_loss: 0.2963 - val_mae: 0.4222 - va
l_mse: 0.2963
Epoch 12/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 20s 73ms/step - loss: 0.6600 - mae: 0.6027 - mse: 0.6600 - val_loss: 1.2769 - val_mae: 0.9865 - va
l_mse: 1.2769
Epoch 13/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 16s 71ms/step - loss: 0.7775 - mae: 0.6934 - mse: 0.7775 - val_loss: 2.6031 - val_mae: 1.5282 - va
l_mse: 2.6031
Epoch 14/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 21s 75ms/step - loss: 1.6116 - mae: 0.9204 - mse: 1.6116 - val_loss: 1.0376 - val_mae: 0.9338 - va
```

```
l_mse: 1.0376
Epoch 15/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 17s 75ms/step - loss: 1.4669 - mae: 0.9627 - mse: 1.4669 - val_loss: 0.2986 - val_mae: 0.4470 - va
l_mse: 0.2986
Epoch 16/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 16s 72ms/step - loss: 0.8356 - mae: 0.7014 - mse: 0.8356 - val_loss: 0.2135 - val_mae: 0.3604 - va
l_mse: 0.2135
Epoch 17/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 21s 74ms/step - loss: 0.8790 - mae: 0.7652 - mse: 0.8790 - val_loss: 0.2506 - val_mae: 0.4028 - va
l_mse: 0.2506
Epoch 18/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 19s 85ms/step - loss: 0.8240 - mae: 0.7061 - mse: 0.8240 - val_loss: 0.2238 - val_mae: 0.3777 - va
l_mse: 0.2238
Epoch 19/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 17s 75ms/step - loss: 0.7877 - mae: 0.7105 - mse: 0.7877 - val_loss: 0.1640 - val_mae: 0.3250 - va
l_mse: 0.1640
Epoch 20/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 15s 68ms/step - loss: 0.6425 - mae: 0.6448 - mse: 0.6425 - val_loss: 0.2106 - val_mae: 0.3730 - va
l_mse: 0.2106
Epoch 21/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 22s 74ms/step - loss: 0.6352 - mae: 0.6110 - mse: 0.6352 - val_loss: 1.7825 - val_mae: 1.2679 - va
l_mse: 1.7825
Epoch 22/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 16s 73ms/step - loss: 0.8551 - mae: 0.7412 - mse: 0.8551 - val_loss: 1.0107 - val_mae: 0.9043 - va
l_mse: 1.0107
Epoch 23/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 20s 70ms/step - loss: 0.9024 - mae: 0.7414 - mse: 0.9024 - val_loss: 0.1590 - val_mae: 0.3201 - va
l_mse: 0.1590
Epoch 24/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 16s 72ms/step - loss: 0.5602 - mae: 0.5820 - mse: 0.5602 - val_loss: 0.1748 - val_mae: 0.3370 - va
l_mse: 0.1748
Epoch 25/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 15s 70ms/step - loss: 0.8428 - mae: 0.7288 - mse: 0.8428 - val_loss: 0.1961 - val_mae: 0.3577 - va
l_mse: 0.1961
Epoch 26/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 21s 71ms/step - loss: 0.5287 - mae: 0.5832 - mse: 0.5287 - val_loss: 1.1978 - val_mae: 1.0268 - va
l_mse: 1.1978
Epoch 27/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 20s 70ms/step - loss: 0.6170 - mae: 0.6154 - mse: 0.6170 - val_loss: 0.2453 - val_mae: 0.4062 - va
l_mse: 0.2453
Epoch 28/30
```

```
219/219 ━━━━━━━━━━━━━━━━━━━━ 21s 73ms/step - loss: 0.5459 - mae: 0.5502 - mse: 0.5459 - val_loss: 0.1524 - val_mae: 0.3171 - va
l_mse: 0.1524
Epoch 29/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 15s 70ms/step - loss: 0.6870 - mae: 0.6362 - mse: 0.6870 - val_loss: 0.1917 - val_mae: 0.3541 - va
l_mse: 0.1917
Epoch 30/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 16s 71ms/step - loss: 0.3331 - mae: 0.4300 - mse: 0.3331 - val_loss: 0.2661 - val_mae: 0.4261 - va
l_mse: 0.2661
```

In [40]:

```
def plot_history(history, key):
  plt.plot(history.history[key])
  plt.plot(history.history['val_'+key])
  plt.xlabel("Epochs")
  plt.ylabel(key)
  plt.legend([key,'val_'+key])
  plt.show()
#plot the history
plot_history(history,'loss')
```



In [41]:
```
z= model.predict(X_test)
```

```
1094/1094 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step
```

In [42]:
```
r2_score(y_test, z)
```

Out[42]:
```
0.9968757564530712
```

In [43]:
```
mse = mean_squared_error(y_test, z)
rmse = mse**.5
print("mse : ",mse)
print("rmse : ",rmse)
print("errors for neural net")
```

```
mae = mean_absolute_error(y_test, z)
print("mae : ",mae)
```

```
mse :  0.26881115292024316
rmse :  0.5184700115920333
errors for neural net
mae :  0.42977989335235917
```

In [44]:
```
from sklearn.metrics import mean_absolute_percentage_error
mean_absolute_percentage_error(y_test, z)
```

Out[44]:
```
0.009424125279191335
```

In [45]:
```
!jupyter nbconvert --to html /content/Porter Delivery Time Estimation.ipynb
```